

Relatório 1ª Fase | Laboratórios de Informática III
Grupo 1 | 2025/2026

Tomás Ferreira (A110182)
Afonso Barros (A112178)
Lucas Pinto (A111442)

Índice

1. Introdução 3
2. Desenvolvimento 3
   2.1. Sistema 3
   2.2. Discussão 5
      2.2.1. Fluxo do Programa 5
      2.2.2. *Queries* 5
      2.2.3. Otimizações 7
      2.2.4. Testes 7
      2.2.5. Dificuldades Sentidas 10
3. Conclusão 10

1. Introdução

Este relatório coincide com a 1ª fase do projeto da Unidade Curricular de Laboratórios de Informática 3 realizado no ano 25/26 na Universidade do Minho.

O principal objetivo deste trabalho consiste em armazenar e organizar dados de forma estruturada, bem como implementar mecanismos que permitam a realização de operações de pesquisa, associação de forma rápida e minimizando o uso de memória. Para tal temos em atenção o encapsulamento, a modularização e a gestão de memória.

2. Desenvolvimento

Este projeto consiste em implementar um sistema que permita ler ficheiros de texto estruturados de forma especificada pelos docentes, *parsing* , criação de entidades e estruturas de dados para armazenar essa informação e ainda a realização de *queries*. Para o funcionamento do programa recebemos como argumentos os caminhos para a pasta onde se encontram os *datasets* e ainda o caminho para o ficheiro que contém *queries* a serem executados. No fim do programa devem ser gerados na pasta resultados um ficheiro para cada *querie*, com o resultado desta.

2.1. Sistema

O nosso sistema foi concebido com o objetivo de garantir a separação de responsabilidades entre os diferentes módulos (modularidade) e assegurar o encapsulamento. O sistema encontra-se dividido em dois componentes principais:

Processamento e Validação dos *Datasets*

Para esta componente, começamos por criar a pasta entidades, definindo as estruturas de dados que as representavam, assim como as respetivas funções de manipulação (*free_entidade*, *getters*, *setters*, *create_entidade*). Esta parte teve o cuidado de manter estas estruturas opacas, ou seja, garantir que apenas se pode manipular os seus campos com as respetivas funções, as quais garantem que não existe partilha de memória com o exterior do módulo.

De seguida, foi criada a pasta gestores, onde foram definidas as estruturas que vão armazenar todas as entidades de um determinado tipo. Optamos ainda por criar um gestor global que armazena os restantes gestores, uma vez que esta abordagem centraliza o acesso a todas as entidades do sistema, facilita a passagem de parâmetros entre módulos e simplifica a gestão de memória.

O módulo *parsers*, utilizando os módulos referidos anteriormente, completa esta componente processando cada linha do ficheiro de dados, fazendo o respetivo *parsing*, validando

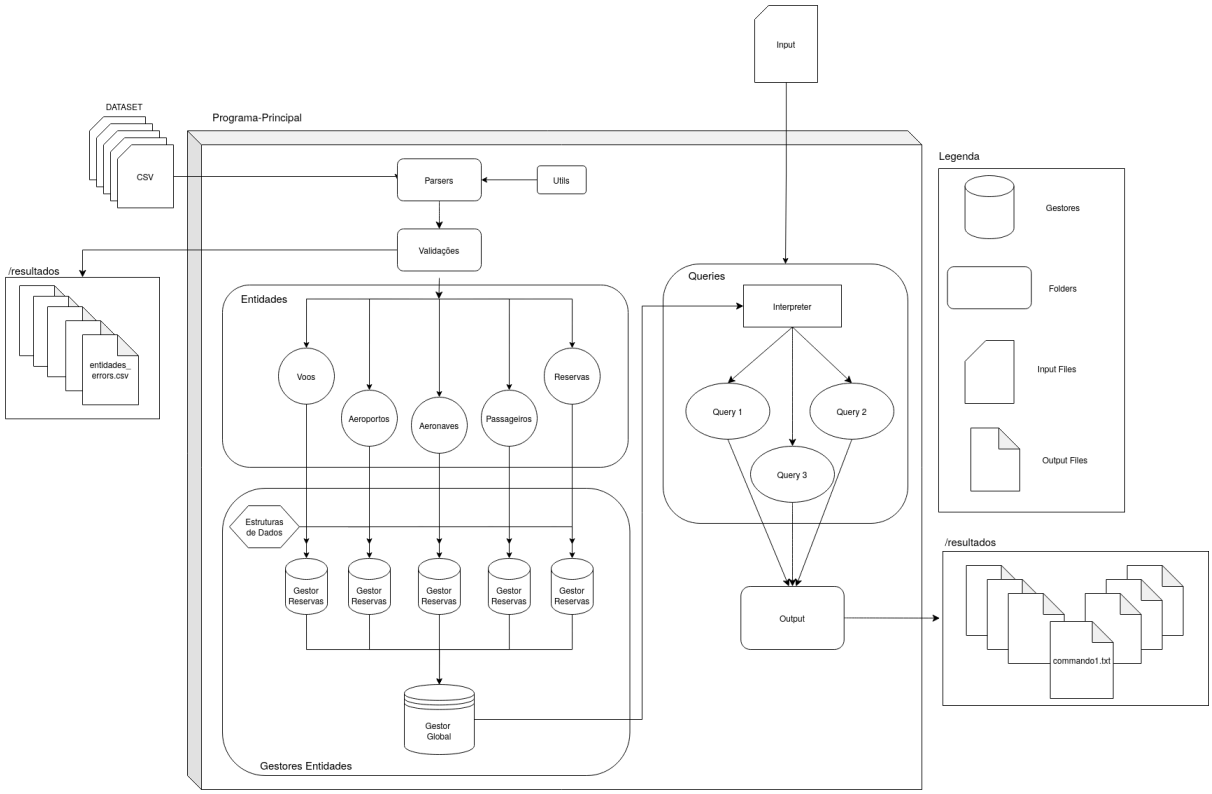
os diferentes campos com recurso ao módulo validações e, no caso de tudo estar válido, utiliza os outros módulos para criar a entidade e inseri-la no respetivo gestor.

Esta fase ocorre apenas no início do programa, lendo cada *dataset* uma única vez para garantir que toda a informação necessária às *queries* fica imediatamente disponível em memória.

Input, *Output* e *Queries*

Dentro do modulo de *queries* temos o interpretador do sistema que recebe o gestor global criado na componente anterior, o qual contém a lógica necessária para percorrer o ficheiro de inputs e chamar a *query* correta. Também é criado o ficheiro commandX.txt, que, através do módulo de *output*, fica com o resultado da *querie*.

Este módulo não interage com os *datasets* — trabalha apenas sobre as estruturas já carregadas em memória, evitando passagens sucessivas por eles.



2.2. Discussão

2.2.1. Fluxo do Programa

A função programa_principal recebe o caminho para o *dataset* e o caminho para as *queries*, sendo responsável por coordenar todo o fluxo de execução do programa.

No início da execução, é criado e inicializado o gestor global, onde estão os gestores para cada tipo de entidade, preparando as estruturas de dados que irão armazenar a informação processada. Estes gestores contêm para cada entidade uma *Hash Table*, que usa como *key* os seus identificadores únicos e como *values* as próprias entidades. Garantindo assim o acesso a uma entidade através do seu *id* em tempo constante (O(1)).

O processamento dos *datasets* utiliza um *parser* geral que recebe uma função de ler a linha da entidade desejada, permitindo assim reutilizar código e consistência no tratamento de todos os ficheiros. A função de fazer *parse* de uma linha de uma entidade recebe o ficheiro de erros, que apenas é aberto uma vez, permitindo a validação e o consequente registo das linhas inválidas.

A ordem do *parse* respeita as dependências entre entidades, seguindo a ordem: aeroportos, aeronaves, voo, passageiros e reservas. A *flag is_used* permite controlar a alocação de memória, evitando a criação de entidades que não são necessárias nesta fase do projeto, mas garantindo simultaneamente a sua validação e o registo de erros.

Concluído o carregamento, o módulo de *queries*, através do *interpreter* lê o ficheiro de comandos linha a linha. Para cada linha, identifica o tipo de *query* através do seu primeiro caractere, extrai os argumentos necessários e invoca a função correspondente (query1, query2, query3).

No final da execução, toda a memória alocada é libertada, através da função *free_gestor_global* definida no gestor global, garantindo assim a ausência de *memory leaks*.

2.2.2. *Queries*

A execução das *queries* tentou encontrar o melhor equilíbrio possível entre o uso de memória e rapidez de execução.

2.2.2.1. *Query* 1

Listar o resumo de um aeroporto, consoante o identificador recebido por argumento.

A realização desta *query* não enfrentou grandes dificuldades visto que o gestor do aeroportos que foi criado previamente continha uma *hash table* com os aeroporto guardados, pelo que, com o identificador conseguimos em tempo constante (O(1)) obter a entidade

pretendida, sendo depois apenas necessário imprimir o resultado pretendido no ficheiro.

2.2.2.2. *Query* 2

Top N aeronaves com mais voos realizados com parâmetro opcional [manufacturer]

A implementação desta *query* procurou minimizar o consumo adicional de memória. A solução passou por adicionar um campo de contagem de voos diretamente na estrutura da entidade aeronave. Durante o *parse* do *dataset* de voos, este contador é incrementado sempre que a aeronave correspondente realiza um voo válido. Para facilitar o acesso e ordenação, foi adicionado um *GPtrArray* ao gestor de aeronaves, preenchido à medida que as entidades são criadas. Esta estrutura armazena apenas apontadores para as entidades existentes, evitando duplicação de dados e evitando um grande aumento no uso de memória. O gestor mantém uma *flag is_sorted* que indica se o *array* de aeronaves já foi ordenado por número de voos decrescente. Esta abordagem permite que a ordenação ocorra apenas na primeira execução da *query* 2. Caso a *query* 2 não seja chamada, evita-se completamente o custo da ordenação.

Para gerar a resposta, o processo depende da presença do filtro de fabricante: quando não existe filtro, percorrem-se simplesmente as primeiras N aeronaves do *GPtrArray* ordenado; quando o filtro está presente, percorre-se o *GPtrArray* sequencialmente até encontrar N aeronaves do fabricante especificado ou até esgotar todas as aeronaves disponíveis. Assim, depois de estar ordenado o *GPtrArray*, no pior caso, apenas percorremos as aeronaves todas (O(A), onde A é o número de aeronaves).

2.2.2.3. *Query* 3

Listar o aeroporto com mais partidas entre 2 datas

Para a *query* 3, adotamos uma estratégia semelhante à da *query* 2. Durante o carregamento do *dataset* de voos, cada data de partida é inserida num *GArray* associado ao aeroporto de origem correspondente. As datas são armazenadas como inteiros no formato AAAAMMDD, o que permite reduzir o espaço ocupado em memória e acelerar as operações de comparação. No gestor de aeroportos, foi criado um *GPtrArray* que facilita a iteração sobre todos os aeroportos e permite ordenar os *arrays* de datas de partida de cada um.

Tal como na *query* 2, o gestor mantém uma *flag is_sorted* que, neste caso, indica se os *arrays* de datas de cada aeroporto já foram ordenados. Desta forma, a ordenação ocorre apenas na primeira invocação da *query* 3, evitando ordenar várias vezes e também evita ordenar quando não é necessário. Com esta abordagem o cálculo do número de voos entre 2 datas fica eficiente: através de pesquisa binária, determina-se a posição da primeira data superior ou igual ao limite inferior do intervalo e a posição da última data inferior ou igual ao limite superior. O número de partidas é então obtido pela diferença entre os índices encontrados, acrescida de uma unidade.

Para identificar o aeroporto com mais partidas, percorremos todos os aeroportos, calculamos o número de partidas no intervalo através do método descrito, e mantemos

registo do aeroporto com o maior valor encontrado. Assim a complexidade desta *query* é O(A . log(D)) onde A é o número de aeroportos e D é o número de datas de partida de cada aeroporto

2.2.3. Otimizações

Foram realizadas várias melhorias tanto ao nível do tempo de execução como da memória utilizada nesta fase. Para reduzir o uso de memória, optou-se por utilizar estruturas de dados que representassem a informação necessária no menor número possível de bytes (por exemplo, converteu-se um *enum* utilizado para o estado de um voo, que ocupava 4 bytes, para um simples *char* de 1 byte). Além disso, evitou-se armazenar dados que não têm utilidade nesta fase do projeto; contudo, como poderão ser necessários na segunda fase, foram apenas comentados em vez de removidos.

Relativamente às entidades reservas e passageiros, que não são utilizadas nesta fase: no caso das reservas, apenas se realiza a validação sem criar qualquer estrutura, enquanto para os passageiros é criada uma *GHashTable* sem estruturas internas (funcionando como um set) usada apenas para validação de outras entidades. Por último, optou-se por não guardar os *IDs* que ligam entidades entre si, mas sim apontadores diretos para as respetivas estruturas.

No que diz respeito às melhorias de desempenho, destaca-se a redução de acessos repetidos a ficheiros (por exemplo, os ficheiros de erros de validação), a criação de funções auxiliares para ignorar argumentos desnecessários durante a leitura do *dataset* (as funções *jump* e *skip*), e a otimização da leitura de strings que apenas podem assumir determinados valores (como o estado de um voo), garantindo que para cada um é necessário, no máximo, um único *strcmp*.

2.2.4. Testes

2.2.4.1. Programa Testes

Para garantir a correção e avaliar o desempenho do sistema foi desenvolvido o programa de testes que armazena e permite obter informações que consideramos relevantes ao longo da execução do programa.

O programa mede o consumo de memória associado ao processamento de cada entidade, permitindo identificar onde se concentra o uso de recursos. No nosso caso, verificamos que o *dataset* de voos é responsável pela maior parte da memória utilizada (aproximadamente 117 MB), enquanto aeroportos e aeronaves ocupam significativamente menos espaço (4.5 MB e 320 KB, respetivamente). Estes valores orientaram as nossas decisões de otimização, focando os esforços nas estruturas que mais impacto têm no consumo total. Optamos por não calcular o espaço utilizado pelo parse dos passageiros uma vez que estas estruturas apenas são guardadas com o objetivo de validação, não tendo qualquer dos seus atributos.

Relativamente aos tempos de parsing, o programa regista a duração do processamento de cada dataset. Os resultados mostram que o parse dos voos domina o tempo total de carregamento (cerca de 2 segundos), enquanto aeroportos e aeronaves são processados em menos de 10 milissegundos. Esta discrepância justifica-se pelo tamanho do dataset e pela complexidade das validações necessárias para os voos.

Além disso, o programa revela o tempo médio de execução de cada query, o número de queries corretas de cada tipo, assim como, no caso de existirem erros, a primeira linha onde eles ocorrem.

No final da execução, o programa apresenta o consumo total de memória e o tempo total decorrido, fornecendo uma visão global do desempenho do programa.

2.2.4.2. Especificações

As máquinas dos 3 elementos do grupo possuem as seguintes especificações:

Computador 1:

- OS: Ubuntu 24.04.2 LTS - kernel 6.14.0-35--generic
- CPU: 13th Gen Intel(R) Core(TM) i7-13700H
- Memoria: 16GB DDR5-4800MHz

Computador 2:

- OS: Pop! OS 22.04 LTS - kernel 6.9.3-76
- CPU: 13th Gen Intel® Core™ i5-1335U
- Memoria: 16GB DDR5-5200MHz

Computador 3:

- OS: EndeavourOs - 6.17.7-arch1-1
- CPU:9th Gen Intel(R) Core(TM) i7-9850H
- Memoria: 16GB DDR4-3200MHz

2.2.4.3. Resultados dos Testes

Para comparar os resultados obtidos entre os 3 computadores, recorremos ao programa de testes desenvolvido. De forma a garantir a fiabilidade das medições, procuramos manter condições controladas, isolando as características do hardware como única variável.

Os testes foram realizados com todos os computadores ligados à corrente, acabados de reiniciar e apenas com o terminal aberto, minimizando programas em segundo plano que pudessem interferir nos resultados.

Utilizamos o *dataset* com erros para todos os testes. Antes das medições finais, executamos o programa 3 vezes em cada máquina para aquecer a cache do processador. Só depois é que realizamos as 3 execuções cujos resultados foram registados nas tabelas abaixo.

Parse	Computador 1 (s)	Computador 2 (s)	Computador 3 (s)
Parser Voos	1.730841	0.957251	1.384883
	1.753316	0.964734	1.321208
	1.779274	0.956015	1.365895
Parser Aeroportos	0.013861	0.011885	0.023310
	0.013948	0.012765	0.025061
	0.014353	0.010123	0.022779
Parser Aeronaves	0.001089	0.001254	0.002408
	0.001429	0.001249	0.002157
	0.001305	0.000815	0.002300
Média Parser Voos	1.754477	0.953333	1.357329
Média Parser Aeroportos	0.014054	0.011591	0.023647
Média Parser Aeronaves	0.001274	0.001106	0.002288

Tabela 1: Tempos de *pare* dos *datasets* nos 3 computadores.

Parser	Computador 1 (KB)	Computador 2 (KB)	Computador 3 (KB)
Parser Voos	117931.3	116742.7	116453
Parser Aeroportos	4609.33	6300	6316
Parser Aeronaves	213.333	217.3333	220.66

Tabela 2: Uso médio de memória durante o *parse* dos *datasets* referente às 3 execuções

Query	Computador 1 (ms)	Computador 2 (ms)	Computador 3 (ms)
Q1	0.064740	0.010835	0.053967
	0.018328	0.008301	0.045501
	0.016998	0.007343	0.036895
Q2	0.053851	0.016196	0.029975
	0.065361	0.019952	0.028753
	0.040465	0.015816	0.032133
Q3	3.041387	1.560759	1.678025
	3.096051	1.495172	1.678925
	3.081276	1.519171	1.574475
Média Q1	0.033355	0.008826	0.045454
Média Q2	0.053226	0.017321	0.030287
Média Q3	3.072905	1.525034	1.6438083

Tabela 3: Tempos de execução das *queries* nos 3 computadores.

	Computador 1	Computador 2	Computador 3
Tempo Total (s)	2.189564	1.185083	1.361208
Memória Total (KB)	132574.7	132642.7	132432

Tabela 4: Tempo médio total de execução e média da memória total utilizada nas 3 execuções.

Verifica-se que os resultados obtidos para os tempos variam bastante consoante o *hardware* que está a executar o programa. Isto pode justificar-se, possivelmente, devido à memória (RAM) que os diferentes processadores possuem. Quanto à utilização de memória na execução do programa, esta foi consistente entre os três computadores (132MB), permitindo perceber que esta variável não depende tanto do *hardware* utilizado.

2.2.5. Dificuldades Sentidas

Durante o desenvolvimento, surgiram dificuldades relacionadas com *memory leaks*, tarefa que por vezes foi bastante desafiante de resolver. Outra dificuldade foi a constante reestruturação do projeto, que implicou adaptar e reorganizar várias partes do código.

3. Conclusão

De um modo geral, consideramos que a primeira parte do projeto foi concluída com sucesso, pois implementamos todo aquilo que foi pedido pelos docentes, passando em todos os testes da plataforma. Ao longo desta fase percebemos a importância de manter o código organizado, apostar numa estrutura modular e ter atenção ao uso da memória e dos restantes recursos. Este trabalho ajudou-nos também a ganhar mais autonomia na resolução de problemas e a planear melhor as diferentes etapas, o que facilitou o cumprimento dos objetivos e permitiu criar uma base sólida para a fase seguinte. Mesmo considerando que esta foi fase realizada com sucesso, sabemos que ainda há aspetos a melhorar, como ao nível da otimização da memória utilizada e da simplificação de algumas partes do código. São pontos que queremos trabalhar de forma mais cuidada na 2.ª fase.